

Project UAS JST TI 23 SE-1, SE-2 dan SH

Panduan Pengerjaan Tugas Akhir Projek

Judul Proyek: "The Zen-Day Optimizer"

Konsep: Sebuah aplikasi asisten harian yang membantu pengguna mengelola energi, fokus, dan rutinitas mereka menggunakan berbagai algoritma ANN untuk tugas yang berbeda

1. Deskripsi Proyek

Mahasiswa diminta membangun aplikasi berbasis Python yang menyelesaikan masalah spesifik dalam kehidupan sehari-hari (seperti manajemen stres, pemilihan makanan, atau pengenalan pola kebiasaan, dll) dengan mengimplementasikan minimal **dua** dari model JST yang telah dipelajari



Contoh Skenario & Model yg Dapat Dipilih

2. Skenario Implementasi Model

Mahasiswa dapat memilih komponen berikut untuk dibangun:

- **Penyaring Notasi Penting (Perceptron/Adaline):**
 - Mengklasifikasikan apakah sebuah pesan masuk "Penting" atau "Bisa Diabaikan" berdasarkan threshold tingkat urgensi dan pengirim
- **Prediksi Mood & Energi (Backpropagation):**
 - Memprediksi tingkat produktivitas pengguna di sore hari berdasarkan data input pagi hari (jam tidur, jumlah kafein, dan cuaca)
- **Pengenalan Angka/Symbol Tangan (Hopfield Network):**
 - Sistem pengaman sederhana di mana pengguna harus menggambar pola tertentu pada matriks 8 x 8 untuk membuka aplikasi (Auto-associative memory)
- **Segmentasi Gaya Hidup (Kohonen/SOM):**
 - Mengelompokkan data aktivitas mingguan pengguna ke dalam cluster "Produktif", "Santai", atau "Burnout" tanpa label sebelumnya
- **Estimasi Kebutuhan Kalori (Radial Basis Function):**
 - Menghitung estimasi kalori yang dibutuhkan secara non-linear berdasarkan aktivitas fisik yang bervariasi.
- Dll

Contoh Skenario & Model yg Dapat Dipilih

3. Daftar Model yang Dapat Dipilih

Pilihlah model yang paling relevan dengan karakteristik masalah Anda:

- **Klasifikasi Linier:** M-P, Hebbian, Perceptron, Adaline
- **Klasifikasi Non-Linier/Kompleks:** Madaline, Backpropagation, Radial Basis Function (RBF)
- **Unsupervised Learning/Clustering:** Kohonen (SOM), Learning Vector Quantization (LVQ), Counter Propagation
- **Memori Asosiatif:** Hopfield Network

Contoh-contoh Tampilan Python

4. Detail Teknis (Python)

Proyek ini wajib menggunakan Python dengan contoh2 asset yg bisa digunakan:

- **Library:** NumPy dan Pandas untuk pengolahan matriks manual (sangat disarankan membangun kelas model dari nol/scratch untuk model dasar seperti Perceptron hingga Madaline)
- **Visualisasi:** Matplotlib atau Seaborn untuk menampilkan kurva *error* (MSE) dan hasil clustering
- **Interface:** Streamlit atau Tkinter untuk membuat dashboard sederhana agar proyek terlihat seperti aplikasi nyata

Struktur Laporan & Demonstrasi

Untuk level perguruan tinggi, penilaian harus mencakup:

1. **Analisis Matematis:** Penjelasan bagaimana bobot (w) diupdate pada setiap iterasi. Misalnya, menunjukkan penggunaan fungsi aktivasi $f(x) = \text{sgn}(x)$ pada Perceptron atau derivatif fungsi sigmoid pada Backpropagation misalnya
2. **Uji Coba Parameter:** Mahasiswa harus menunjukkan perbandingan hasil jika learning rate (η) diubah atau jika jumlah hidden layer ditambah
3. **Demo Live:** Menunjukkan aplikasi menangani input data baru secara real-time

Laporan harus disusun secara akademis dengan poin-poin berikut:

- **Bab I: Latar Belakang:** Mengapa masalah ini penting diselesaikan?
- **Bab II: Arsitektur Model:** Penjelasan jumlah *input*, *hidden*, dan *output layer*, serta fungsi aktivasi dan model apa yang digunakan
- **Bab III: Perhitungan Matematis:** Tunjukkan contoh perhitungan satu iterasi (Forward & Backward) secara manual atau dalam cuplikan kode
- **Bab IV: Analisis Hasil:** Bagaimana pengaruh perubahan *learning rate* (η) atau jumlah *epoch* terhadap akurasi?
- **Bab V: Kesimpulan:** Apakah model berhasil menyelesaikan masalah harian tersebut?
- **LAMPIRAN (APPENDIX):**
 - **Full Source Code:** Lampirkan kode lengkap python (pastikan ada komentar/comment di setiap bagian kode)
 - **Library tambahan yang dipakai**

Sistem Penilaian

1. Individual proyek mendapatkan nilai bonus 10 pts apabila aplikasinya berjalan dengan baik
2. Penilaian tertinggi akan ditentukan berdasarkan proyek terbaik dengan ide yang paling kreatif (baik individu ataupun tim). Penilaian lainnya akan mengikuti berjenjang sampai dengan nilai terendah
3. Proyek akan dinilai apabila memenuhi kriteria berikut:
 1. Selesai sebelum deadline
 2. Aplikasi berjalan dengan baik saat presentasi
 3. Ada dokumentasi lengkap logika dibalik aplikasi tersebut, model jst yang digunakan (untuk menangani masalah apa, cara kerjanya, logikanya, dll) dan penjelasan perintah2 phyton yang digunakan (cara kerja, hasil yang diharapkan, inputnya apa, dll)
4. Bisa tambahkan user interface sederhana atau menarik (untuk nilai plus) sebagai tampilan aplikasi

Kriteria	Bobot	Detail
Kreativitas Solusi	20%	Seberapa unik masalah harian yang diangkat
Kualitas Kode	30%	Kebersihan kode, penggunaan NumPy, dan logika algoritma
Analisis Matematis	30%	Kedalaman pemahaman tentang <i>update weights</i> dan <i>error signal</i>
Presentasi & Demo	20%	Kemampuan menjelaskan cara kerja aplikasi secara <i>live</i>

Format Presentasi dan Dokumentasi

Presentasi:

1. Waktu presentasi antara 4-7 menit per proyek (rekam presentasi kalian dengan format video umum dengan hp dan kumpulkan ke email. Karena waktu terbatas presentasi kelas hanya random 15 orang)
2. Bisa menggunakan PPT atau format lain untuk presentasi
3. Cantumkan gambar2 untuk menjelaskan alur kerja aplikasi tersebut

Dokumentasi:

1. Format bisa word document atau PDF
 2. Cantumkan penjelasan ide kreatifnya dan tujuan aplikasi tersebut
 3. Model2 JST apa yang digunakan, alasan pemilihannya dan penjelasan perannya dalam aplikasi anda
 4. Penjelasan input, output dan error message yang mungkin dihasilkan
- Deadline Pengumpulan Proyek 1 Minggu sebelum Jadwal UAS
 - Penamaan file dan folder: Nama_NIM_UAS.PDF dan folder Nama_NIM_Kelas. Sertakan code python-nya di dalam file dan rekaman presentasi dengan low res small size saja supaya tidak terlalu besar - maksimum 30 MB file presentasinya
 - Upload Proyek Akhir ke google drive di vickyindrawan@global.ac.id (alamat menyusul)

Contoh Output Kreatif: "The Smart Snack Picker"

Jika mahasiswa memilih LVQ (Learning Vector Quantization):

- **Masalah:** Pengguna bingung memilih camilan saat lembur
- **Solusi:** Algoritma dilatih dengan *prototype* camilan (Sehat vs Junk Food). Berdasarkan input "Tingkat Lapar" dan "Waktu Tersisa", LVQ akan mengarahkan pengguna ke kategori camilan yang paling sesuai dengan target kesehatan mereka

Contoh Code Phyton salah satu opsi di Slide 3

```
import numpy as np

# 1. Definisi Fungsi Aktivasi Sigmoid & Turunannya
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# 2. Dataset Sederhana (Input: [Jam Tidur, Cangkir Kopi])
# Skala data 0-1 (Normalisasi)
# Contoh: 8 jam tidur = 0.8, 2 cangkir kopi = 0.2
inputs = np.array([[0.8, 0.2], [0.4, 0.1], [0.3, 0.6], [0.7, 0.5]])
# Output: [1] = Bugar, [0] = Lelah
expected_output = np.array([[1], [0], [0], [1]])

# 3. Inisialisasi Parameter
epochs = 10000
lr = 0.1
inputLayerNeurons, hiddenLayerNeurons, outputLayerNeurons = 2, 3, 1

# Bobot dan Bias Acak
hidden_weights = np.random.uniform(size=(inputLayerNeurons, hiddenLayerNeurons))
hidden_bias = np.random.uniform(size=(1, hiddenLayerNeurons))
output_weights = np.random.uniform(size=(hiddenLayerNeurons, outputLayerNeurons))
output_bias = np.random.uniform(size=(1, outputLayerNeurons))
```

```
# 4. Training Loop (Backpropagation)
for _ in range(epochs):
    # -- Forward Propagation --
    hidden_layer_activation = np.dot(inputs, hidden_weights) + hidden_bias
    hidden_layer_output = sigmoid(hidden_layer_activation)

    output_layer_activation = np.dot(hidden_layer_output, output_weights) + output_bias
    predicted_output = sigmoid(output_layer_activation)

    # -- Backpropagation --
    error = expected_output - predicted_output
    d_predicted_output = error * sigmoid_derivative(predicted_output)

    error_hidden_layer = d_predicted_output.dot(output_weights.T)
    d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_layer_output)

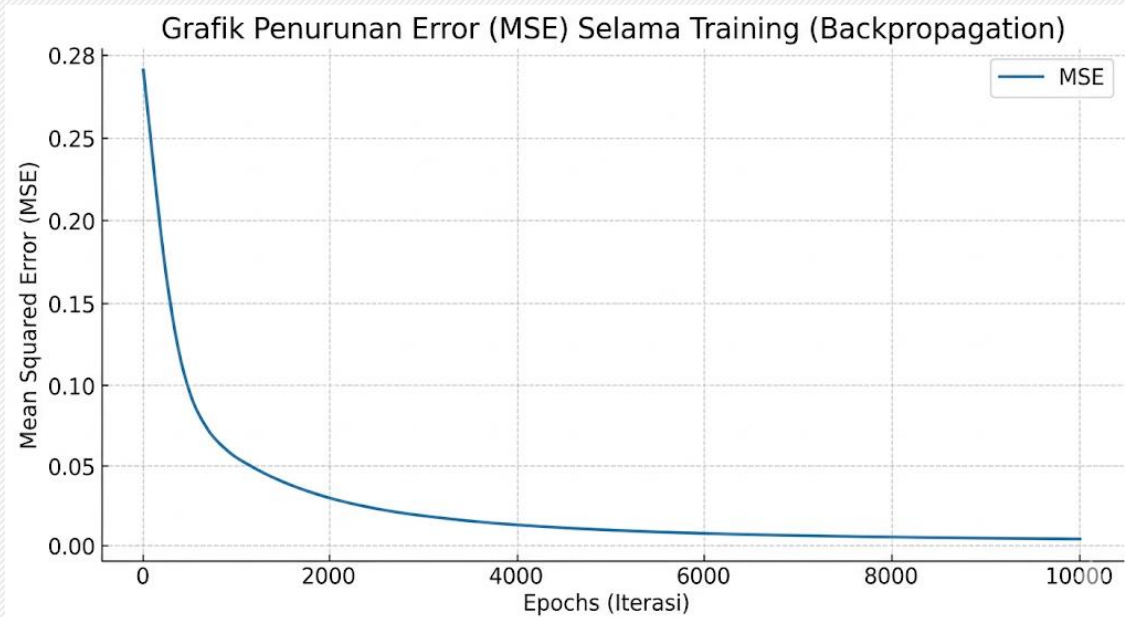
    # -- Updating Weights & Biases --
    output_weights += hidden_layer_output.T.dot(d_predicted_output) * lr
    output_bias += np.sum(d_predicted_output, axis=0, keepdims=True) * lr
    hidden_weights += inputs.T.dot(d_hidden_layer) * lr
    hidden_bias += np.sum(d_hidden_layer, axis=0, keepdims=True) * lr

print("Hasil Prediksi Setelah Training:")
print(predicted_output)
```

Contoh Penggunaan Matplotlib untuk Tampilan Grafik

Contoh Implementasi Kode

Berikut adalah potongan kode yang bisa ditambahkan ke dalam *training loop* Backpropagation atau Adaline



```
import matplotlib.pyplot as plt
import numpy as np
```

```
# Inisialisasi list untuk menyimpan log error
history_loss = []
```

```
# Diasumsikan dalam loop training:
```

```
for epoch in range(epochs):
```

```
    # ... proses forward & backward propagation ...
```

```
    # Hitung MSE untuk epoch saat ini
```

```
    mse = np.mean(np.square(expected_output - predicted_output))
```

```
    history_loss.append(mse)
```

```
# --- BAGIAN VISUALISASI ---
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(history_loss, color='blue', linewidth=2)
```

```
plt.title('Grafik Penurunan Error (MSE) Selama Training')
```

```
plt.xlabel('Epoch ke-')
```

```
plt.ylabel('Mean Squared Error')
```

```
plt.grid(True, linestyle='--', alpha=0.6)
```

```
plt.show()
```

Contoh Penggunaan Matplotlib untuk Tampilan Grafik

Logika Penyimpanan Data Error

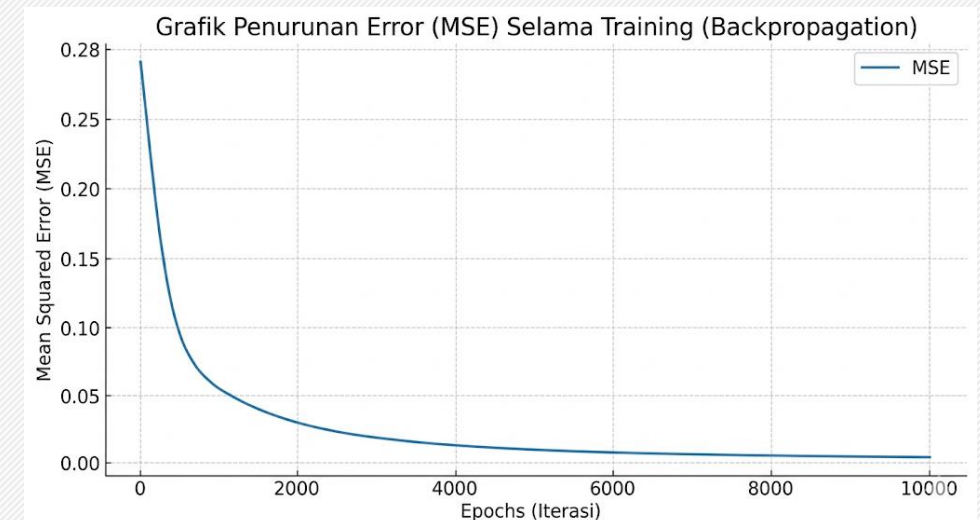
- Mahasiswa perlu membuat sebuah list kosong (misalnya `history_loss`) untuk menyimpan nilai Mean Squared Error (MSE) pada setiap iterasi atau epoch.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_{true} - y_{pred})^2$$

Analisis Grafik (Untuk Contoh Laporan):

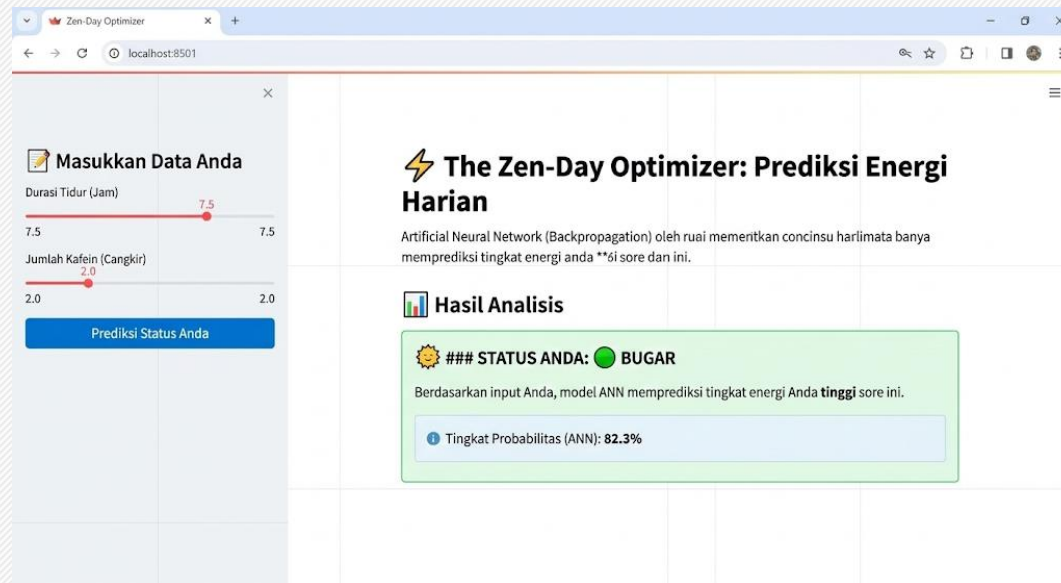
Mahasiswa dapat menggunakan contoh grafik sebagai referensi untuk analisa hasil proyek mereka:

- Fase Awal (Epoch 0 - 1000):** Terlihat penurunan error yang sangat drastis (garis biru menekuk tajam). Ini menunjukkan model ANN sedang melakukan penyesuaian bobot besar-besaran untuk mengenali pola dasar data
- Fase Pembelajaran Melandai (Epoch 2000 - 6000):** Penurunan error mulai melambat. Model sudah mulai "memahami" pola data dan sedang melakukan *fine-tuning* pada bobot-bobotnya
- Fase Konvergensi (Epoch > 8000):** Garis grafik terlihat hampir mendatar (stabil) di angka yang sangat rendah (mendekati 0.005). Ini menandakan model sudah mencapai titik optimal dan proses *training* bisa dihentikan karena menambah iterasi tidak lagi menurunkan *error* secara signifikan.



Contoh Penggunaan Streamlit untuk User Interface

Contoh Implementasi Kode Menggunakan Streamlit
Berikut adalah potongan kode untuk user interface ini



Pastikan sudah install berikut:

pip install streamlit opencv-python numpy matplotlib scikit-image

Contoh Penggunaan Streamlit untuk User Interface

```
import streamlit as st
import numpy as np
import time

# --- BAGIAN 1: DEFINISI MODEL ANN BACKPROPAGATION (MATRIKS SAJA) ---
class TinyBackpropANN:
    def __init__(self, input_size, hidden_size, output_size):
        # Bobot dan Bias Acak (untuk latihan, kita latih di awal)
        self.w1 = np.random.uniform(size=(input_size, hidden_size))
        self.b1 = np.random.uniform(size=(1, hidden_size))
        self.w2 = np.random.uniform(size=(hidden_size, output_size))
        self.b2 = np.random.uniform(size=(1, output_size))

        # Contoh dataset latihan (sama seperti contoh sebelumnya)
        self.X = np.array([[0.8, 0.2], [0.4, 0.1], [0.3, 0.6], [0.7, 0.5]]) # Sleep, Coffee
        self.y = np.array([[1], [0], [0], [1]]) # Status: Bugar, Lelah

    def sigmoid(self, x): return 1 / (1 + np.exp(-x))
    def sigmoid_derivative(self, x): return x * (1 - x)
```

```
def train(self, epochs=10000, lr=0.1):
    for _ in range(epochs):
        # -- Forward Prop --
        hidden_layer_activation = np.dot(self.X, self.w1) + self.b1
        hidden_layer_output = self.sigmoid(hidden_layer_activation)
        output_layer_activation = np.dot(hidden_layer_output, self.w2) + self.b2
        predicted_output = self.sigmoid(output_layer_activation)

        # -- Backprop --
        error = self.y - predicted_output
        d_predicted_output = error * self.sigmoid_derivative(predicted_output)
        error_hidden_layer = d_predicted_output.dot(self.w2.T)
        d_hidden_layer = error_hidden_layer * self.sigmoid_derivative(hidden_layer_output)

        # -- Update Weights & Biases --
        self.w2 += hidden_layer_output.T.dot(d_predicted_output) * lr
        self.b2 += np.sum(d_predicted_output, axis=0, keepdims=True) * lr
        self.w1 += self.X.T.dot(d_hidden_layer) * lr
        self.b1 += np.sum(d_hidden_layer, axis=0, keepdims=True) * lr

def predict(self, input_data):
    # Forward propagation untuk 1 data baru
    hidden_layer_activation = np.dot(input_data, self.w1) + self.b1
    hidden_layer_output = self.sigmoid(hidden_layer_activation)
    output_layer_activation = np.dot(hidden_layer_output, self.w2) + self.b2
    predicted_output = self.sigmoid(output_layer_activation)
    return predicted_output
```


Contoh Penggunaan Streamlit untuk User Interface

```
# --- BAGIAN 2: MEMBANGUN APLIKASI WEB DENGAN STREAMLIT ---
```

```
# 1. Judul dan Deskripsi
```

```
st.set_page_config(page_title="Zen-Day Optimizer", page_icon="⚡")
```

```
st.title("⚡ The Zen-Day Optimizer: Prediksi Energi Harian")
```

```
st.markdown("""
```

```
Aplikasi ini menggunakan algoritma **Artificial Neural Network (Backpropagation)** untuk  
memprediksi tingkat energi Anda di sore hari berdasarkan data pagi hari.
```

```
""")
```

```
# 2. Persiapan Model ANN (Training di Awal)
```

```
# Menggunakan st.cache_resource agar model tidak dilatih ulang setiap kali UI berubah
```

```
@st.cache_resource
```

```
def get_trained_model():
```

```
    model = TinyBackpropANN(input_size=2, hidden_size=3, output_size=1)
```

```
    with st.spinner('Sedang melatih model ANN Backpropagation...'):
```

```
        model.train(epochs=20000) # Latih lebih banyak agar akurat
```

```
    st.success('Model ANN siap digunakan!')
```

```
    return model
```

```
my_ann = get_trained_model()
```

```
# 3. Sidebar untuk Input Pengguna
```

```
st.sidebar.header("📝 Masukkan Data Anda")
```

```
st.sidebar.markdown("Silakan atur data aktivitas Anda hari ini.")
```

```
# Input slider untuk tidur dan kopi
```

```
sleep_duration = st.sidebar.slider("Durasi Tidur (Jam)", 0.0, 12.0, 7.5, step=0.5)
```

```
caffeine_intake = st.sidebar.slider("Jumlah Kafein (Cangkir)", 0.0, 5.0, 2.0, step=0.5)
```

```
# Normalisasi Input (Sama seperti training, ke rentang 0-1)
```

```
norm_sleep = sleep_duration / 12.0
```

```
norm_caffeine = caffeine_intake / 5.0
```

```
# Tombol untuk memicu prediksi
```

```
predict_btn = st.sidebar.button("Prediksi Status Anda")
```

Contoh Penggunaan Streamlit untuk User Interface

4. Bagian Tampilan Hasil Prediksi

st.subheader("🇮🇩 Hasil Analisis")

if predict_btn:

with st.spinner('Menganalisis data dengan model ANN...'):

time.sleep(1) # Efek loading agar realistis

Masukkan input data baru ke model (pastikan dalam format matriks NumPy)

new_data = np.array([[norm_sleep, norm_caffeine]])

prediction = my_ann.predict(new_data)

probabilitas = prediction[0][0] * 100

5. Visualisasi Hasil Berdasarkan Probabilitas

if probabilitas >= 50:

st.balloons() # Efek animasi

st.success("### STATUS ANDA: 🟢 BUGAR")

st.write(f"Berdasarkan input Anda, model ANN memprediksi tingkat energi Anda **tinggi** sore ini.")

st.info(f"Tingkat Probabilitas (ANN): {probabilitas:.1f}%")

else:

st.error("### STATUS ANDA: 🟡 LELAH")

st.write(f"Berdasarkan input Anda, model ANN memprediksi tingkat energi Anda **rendah** sore ini.")

st.info(f"Tingkat Probabilitas (ANN): {probabilitas:.1f}%")

Petunjuk penggunaan jika tombol belum ditekan

else:

st.warning("Silakan atur data aktivitas Anda di **sidebar sebelah kiri** dan klik **Prediksi Status Anda** untuk melihat hasil analisis.")

Contoh Penggunaan Streamlit untuk User Interface

Step-by-Step Cara Kerja Kode

Respons ini menjelaskan bagaimana Python, NumPy, dan Streamlit bekerja bersama untuk membuat aplikasi web yang didukung JST

Langkah 1: Persiapan Lingkungan dan Impor

Kode dimulai dengan mengimpor library yang diperlukan: `streamlit` untuk membuat UI web, `numpy` untuk perhitungan matriks ANN dari awal, dan `time` untuk efek loading sederhana.

Langkah 2: Definisi Kelas ANN Backpropagation

Ini adalah salinan inti algoritma Backpropagation kita. Perbedaannya di sini adalah data dikemas dalam kelas `TinyBackpropANN`

- Saat kelas dibuat (`__init__`), bobot diinisialisasi secara acak dan dataset sampel disiapkan
- Fungsi `train` melakukan perulangan (forward & backward propagation) ribuan kali untuk menginisialisasi bobot yang valid

Contoh Penggunaan Streamlit untuk User Interface

Langkah 3: Konfigurasi dan Training Model di Streamlit

- `st.set_page_config` dan `st.title` membuat judul halaman yang terlihat di browser
- `@st.cache_resource` adalah fitur canggih Streamlit

Fungsi ini memastikan model ANN dilatih **hanya sekali** saat aplikasi pertama kali dijalankan. Bobot yang sudah dilatih disimpan di memori, sehingga aplikasi tidak menjadi lambat saat pengguna menggeser slider.

Langkah 4: Membangun Antarmuka Input Pengguna (Sidebar)

Streamlit sangat efisien dalam membuat elemen UI:

- `st.sidebar.slider` membuat slider interaktif di bilah sisi kiri untuk "Jam Tidur" dan "Jumlah Kafein"
- `st.sidebar.button` membuat tombol "Prediksi Status Anda"

Streamlit bertindak sebagai *event listener*; saat tombol ditekan, kode akan mengeksekusi logika prediksi

Contoh Penggunaan Streamlit untuk User Interface

Langkah 5: Normalisasi dan Logika Prediksi

Ini adalah jembatan antara input pengguna dan ANN:

- Input pengguna (misalnya, "7.5 Jam Tidur") harus **dinormalisasi** ke rentang yang sama dengan data training (0-1). Kita membaginya dengan batas maksimum yang masuk akal (Tidur / 12, Kopi / 5)
- Data yang sudah dinormalisasi dikonversi menjadi matriks NumPy `[[norm_sleep, norm_caffeine]]`
- Fungsi `my_ann.predict` dipanggil, yang melakukan satu kali forward propagation menggunakan bobot yang sudah terlatih untuk menghasilkan probabilitas (nilai antara 0-1).

Langkah 6: Menampilkan Hasil Berdasarkan Threshold

Setelah menerima probabilitas dari model, kode memutuskan status energi:

- Jika probabilitas ≥ 0.5 (50%), maka pengguna dianggap "**Bugar**". Hasil ditampilkan menggunakan `st.success` (kotak hijau) dan animasi balon (`st.balloons()`)
- Jika probabilitas < 0.5 , maka pengguna dianggap "**Lelah**". Hasil ditampilkan menggunakan `st.error` (kotak merah)

Contoh Penggunaan Streamlit untuk User Interface

Cara Menjalankan Aplikasi:

1. Pastikan `streamlit` sudah terinstal: `pip install streamlit numpy`
2. Buka terminal/command prompt di folder tempat Anda menyimpan `app.py`
3. Jalankan perintah: `streamlit run app.py`

Beberapa Contoh Small Apps yang Diharapkan

1. **Mood Color Clustering (Kohonen/SOM):** Mengelompokkan perasaan pengguna ke dalam palet warna (Biru untuk tenang, Merah untuk stres) berdasarkan jurnal harian singkat
2. **Burnout Early Warning (Adaline):** Memberikan peringatan dini jika pola aktivitas menunjukkan tanda-tanda kelelahan mental yang linear
3. **Smart Snack Picker (LVQ):** Memilih antara "Buah" atau "Cokelat" berdasarkan tingkat lapar dan durasi belajar
4. **Healthy Food Classifier (Perceptron):** Klasifikasi sederhana apakah menu makan siang hari ini "Zen-Friendly" atau "High-Sugar"
5. **Sleep Quality Scorer (Madaline):** Menilai kualitas tidur berdasarkan variabel suhu kamar, kebisingan, dan jam tidur
6. **Task Prioritizer (Hebbian Learning):** Belajar memperkuat hubungan antara jenis tugas tertentu dengan waktu penyelesaian tercepat pengguna
7. **Zen Space Recommender (RBF):** Merekomendasikan lokasi (Perpustakaan, Taman, atau Kamar) berdasarkan cuaca dan tingkat keramaian
8. **Zen Doodle Analyzer (Hopfield):** Mengenali pola coretan (doodle) favorit pengguna saat mereka sedang bosan atau berpikir keras.
9. **Social Battery Predictor (Backpropagation):** Memprediksi berapa lama pengguna bisa bertahan di acara sosial sebelum merasa lelah (introvert assistant)
10. DII